



UNIVERSITI MALAYSIA TERENGGANU

**FACULTY OF COMPUTER SCIENCE AND
MATHEMATICS**

SEMESTER 4 SESSION 25/26

CSE3433 – SOFTWARE ARCHITECTURE

PROJECT TITLE:

**MICROSERVICE-BASED DISASTER RELIEF COORDINATION
PLATFORM (DRCP)**

MATRIC NO.	NAME
S74307	AHMAD SHAHIRAN AMSYAR BIN SHAFEIN
S76928	PETER LOURDES FERNANDEZ
S76214	MUHAMMAD HAZIM IHSAN BIN ADNAN

Table of Contents

1.0 SYSTEM OVERVIEW	2
2.0 FUNCTIONAL REQUIREMENT.....	2
2.1 USER MANAGEMENT	2
2.2 INCIDENT REPORTING	3
2.3 RESOURCE TRACKING.....	3
2.4 NOTIFICATION DELIVERY	3
2.5 VOLUNTEER COORDINATION.....	4
3.0 NON-FUNCTION REQUIREMENT	5
3.1 FAULT TOLERANCE	5
3.2 SCALABILITY	5
3.3 INTEROPERABILITY	6
3.4 MAINTAINABILITY.....	6
4.0 ARCHITECTURE STYLE	7
5.0 SERVICE IDENTIFICATION	8
6.0 COMMUNICATION DESIGN	8
7.0 ARCHITECTURE DIAGRAM.....	10
7.1 SYSTEM CONTEXT DIAGRAM.....	10
7.2 COMPENANT SERVICE ARCHITECTURE DIAGRAM.....	10
7.3 INTERACTION DIAGRAM.....	11
7.4 DEPLOYMENT DIAGRAM	11
8.0 TECHNOLOGY STACK.....	12
9.0 PROTOTYPE IMPLEMENTATION.....	12
CONCLUSION.....	14
REFLECTION.....	15

1.0 SYSTEM OVERVIEW

The Disaster Relief Coordination Platform (DRCP) is a microservice-based system designed to support efficient disaster response and management. It enables real-time incident reporting, resource allocation, volunteer coordination, and communication among stakeholders such as victims, responders, and administrators. The system ensures high availability, scalability, and seamless integration with external services.

2.0 FUNCTIONAL REQUIREMENT

2.1 USER MANAGEMENT

The User Management Service is responsible for handling user registration, authentication, and access control within the system.

- The system shall allow users to register using an email address or phone number and create secure login credentials.
- The system shall authenticate users during login and provide secure access using role-based access control.
- The system shall support multiple user roles, including Administrator, Coordinator, Volunteer, and Donor.
- The system shall allow users to manage and update their personal profiles, including contact information and passwords.
- The system shall provide account verification and password recovery mechanisms to enhance security.
- The system shall allow administrators to manage user accounts, including updating roles, suspending accounts, or deleting users.

2.2 INCIDENT REPORTING

The Incident Reporting Service enables users to report and manage disaster-related incidents. The system shall allow authorized users to create new incident reports by providing details such as incident type, location, time, and

description. It shall support the uploading of media files, including images and videos, to provide additional context for the reported incident.

The system shall classify incidents based on severity levels and allow authorized users to update the status of incidents as they progress. It shall also provide functionality to search and filter incidents based on various criteria such as location, type, and severity. Furthermore, the system shall ensure that incident information is updated in real-time to support timely decision-making during disaster response.

2.3 RESOURCE TRACKING

The Resource Tracking Service manages the availability and distribution of disaster relief resources. The system shall allow coordinators to add, update, and manage different types of resources, including food, water, and medical supplies. It shall maintain a real-time inventory of available resources and ensure that resource quantities are accurately tracked.

The system shall enable the allocation of resources to specific incidents and track their movement and delivery status from source to destination. It shall generate alerts when resource levels fall below predefined thresholds to prevent shortages. Additionally, the system shall allow users to request resources and record resource usage for reporting and auditing purposes.

2.4 NOTIFICATION DELIVERY

The Notification Delivery Service is responsible for delivering alerts and updates to users across the platform. The system shall generate notifications based on events such as new incident reports, resource updates, and task assignments. It shall support multiple notification channels, including in-app notifications, email, and SMS, to ensure effective communication.

The system shall deliver notifications in real-time or near real-time and allow users to configure their notification preferences based on their needs. It shall also support targeted notifications by considering user roles and locations. Furthermore, the system shall maintain a history of notifications and track their delivery status to ensure reliability.

2.5 VOLUNTEER COORDINATION

The Volunteer Coordination Service manages volunteer participation and task assignments during disaster response. The system shall allow users to register as volunteers and maintain profiles that include their skills, availability, and location. It shall enable coordinators to assign tasks to volunteers based on their skills and proximity to the incident location.

The system shall allow volunteers to accept or reject assigned tasks and update the status of their tasks as they progress. It shall track task completion and provide real-time updates to coordinators. Additionally, the system shall support communication between coordinators and volunteers and monitor volunteer performance to ensure effective coordination during disaster operations.

3.0 NON-FUNCTION REQUIREMENT

3.1 FAULT TOLERANCE

- The system shall be designed to continue operating even if one or more components fail.
- The microservice architecture shall ensure that the failure of a single service does not affect the entire system.
- Each microservice shall operate independently and be isolated from other services to prevent system-wide failure.
- The system shall implement error-handling and retry mechanisms to manage service failures.
- The system shall provide fallback responses to maintain essential functionality during failures.
- The system shall use persistent storage and message queues to prevent data loss.
- The system shall support automatic recovery or restart of failed services whenever possible.

3.2 SCALABILITY

- The system shall be designed to support growth in users, incidents, and data without performance degradation.
- Each microservice shall be independently scalable based on demand.
- The system shall support horizontal scaling by adding more instances of services during high load.
- Load balancing mechanisms shall be used to distribute requests evenly across services.
- The system shall maintain stable performance during peak disaster situations with high user traffic.

3.3 INTEROPERABILITY

The system shall support seamless communication and data exchange between all microservices using RESTful APIs or message-based communication. Standard data formats such as JSON shall be used to ensure consistency and compatibility between services. The system shall also support integration with external systems such as government disaster management platforms, emergency response APIs, or third-party mapping services. The architecture shall ensure that changes in one service do not negatively impact other services, maintaining loose coupling and effective interoperability.

3.4 MAINTAINABILITY

- The system shall be designed to allow easy maintenance, updates, and enhancements.
- Each microservice shall operate independently to allow modifications without affecting other services.
- The system shall follow a modular architecture with clearly defined responsibilities.
- The system shall provide well-documented APIs to support debugging and future development.
- The system shall support API versioning to ensure backward compatibility during updates.
- The system shall include logging and monitoring mechanisms for issue detection and troubleshooting.
- The system shall allow new features or services to be added with minimal changes to existing components.

4.0 ARCHITECTURE STYLE

The architecture style we use for Disaster Relief coordination Platform (DRCP) is Microservice Architecture. Instead of building a massive and monolithic application, the approach we choose will break down the system into small and independence service.

We use this approach because the disaster relief requires extreme reliability and the ability to operate continuously without trouble. This system will handle sudden traffic spike or any sudden problem without broke the entire system.

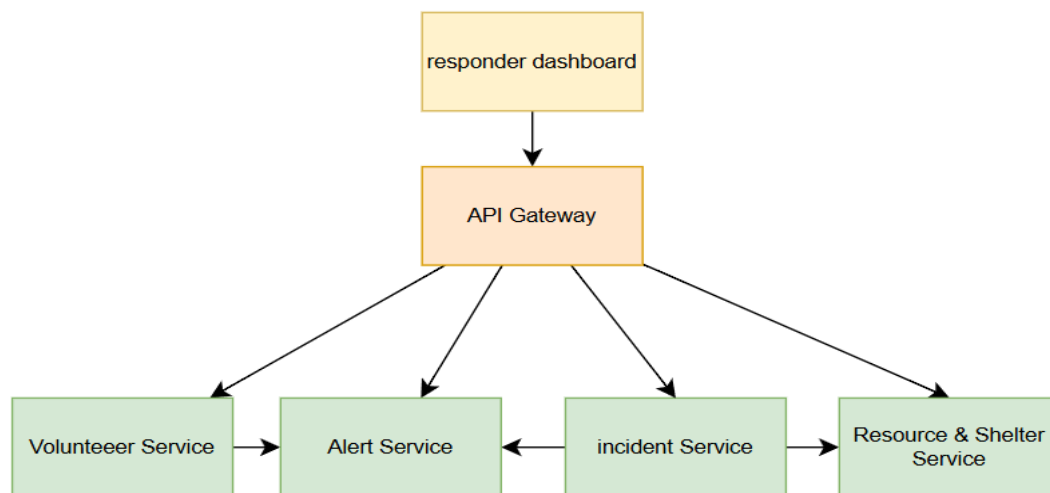
1. **Fault Tolerance and Safe Isolation:** Each service runs in its own isolated sandbox container. If one service experiences a bug or crashes, it does not bring down the rest of the system. The platform maintains its operational capacity even if a single component becomes entirely nonfunctional.
2. **Independent Scalability:** Microservices prevent system-wide crashes caused by sudden user surges during an emergency. Critical components, such as the Alert System, can scale up instantly to handle emergency loads, while less critical services like Donation History remain unaffected
3. **Maintainability Without Downtime:** This decentralized architecture eliminates deployment bottlenecks. Developers can update specific modules such as adding a new resource type without having to take the entire disaster relief platform offline
4. **Containerization Foundation:** The entire architecture is built on containerization, ensuring the code runs uniformly across environments while efficiently sharing the host operating system's kernel
5. **Strict Data Separation:** Data ownership requires a "one service = one database/schema" implementation. This ensures true independence and explicitly avoids the pitfalls of shared databases or cross-service database access
6. **Decoupled Communication:** The isolated services interact strictly via REST or event-driven calls. The design deliberately avoids tight coupling via internal calls to maintain resilience.

5.0 SERVICE IDENTIFICATION

Component	Responsibility	Exclusion
API Gateway	Manages incoming requests from responder dashboards and routes them to the appropriate backend services.	Does not execute business logic, process data, or own a database.
Identity & User Manager Service	Handles role-based access control and authentication for users, including Victims, Rescuers, and Administrators.	Does not process real-time SOS signals or manage physical resource data
Incident Service	Processes real-time SOS signals, complete with GPS coordinates and uploaded photographic evidence.	Does not dispatch automated safety alerts or match volunteers to incident tasks.
Volunteer Service	Executes the logic for matching available volunteers to specific tasks based on their individual skills and geographical proximity to the work site.	Does not manage warehouse supply inventories or authenticate new user registrations.
Resource Service	Manages databases for warehouses and tracks the movement of critical supplies like food, water, and medical equipment	Does not manage volunteer locations or monitor relief camp/shelter capacities.
Shelter Service	Manages the databases and capacity tracking specifically for relief camps	Does not track the physical movement of external warehouse supplies.
Alert Service	Manages the distribution of safety updates and evacuation orders across SMS, Push notifications, and Email channels.	Does not manage incoming requests from responder dashboards or process raw SOS coordinates.

6.0 COMMUNICATION DESIGN

To maintain true fault tolerance and independent scalability, the Disaster Relief Coordination Platform (DRCP) employs a decentralized communication strategy. This architecture explicitly avoids tight coupling via direct internal calls. Instead, services communicate asynchronously via REST APIs and event-driven messaging.

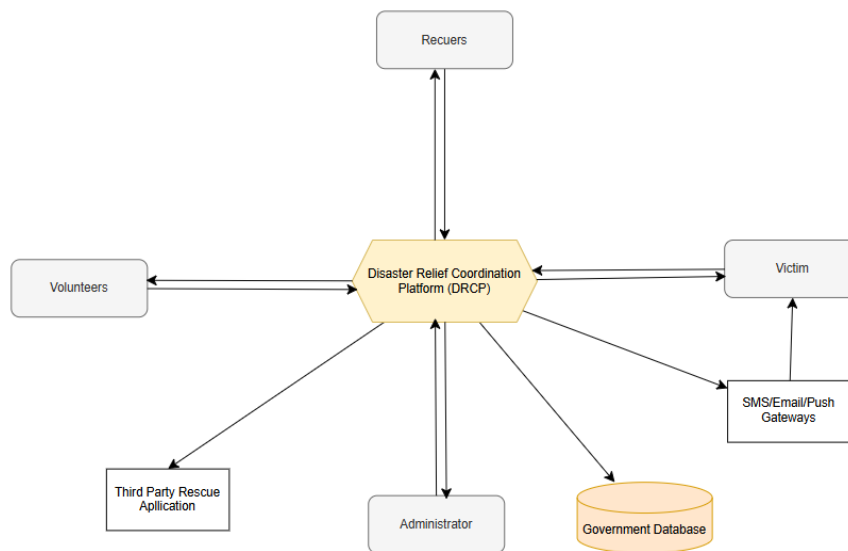


DATA OWNERSHIP

Service Name	The Database	Data Owned & Managed
Identity & User Management Service	Identity Database	User credentials authentication tokens, and role definition (Victim, Rescuer & Admin)
Incident Service	Incident Database	SOS signal logs, GPS coordinates, timestamp, And metadata for uploaded photographic evidence
Volunteer Service	Volunteer Database	Volunteer profiles, skill sets, current availability status, and geographical proximity metrics.
Resource Service	Resource Database	Warehouse locations and inventory levels for food, water, and medical supplies.
Shelter Service	Shelter Database	Relief camp locations, current occupancy, and maximum capacity tracking.
Alert Service	Alert Database	Notification templates, delivery status logs, and historical records of dispatched SMS/Emails

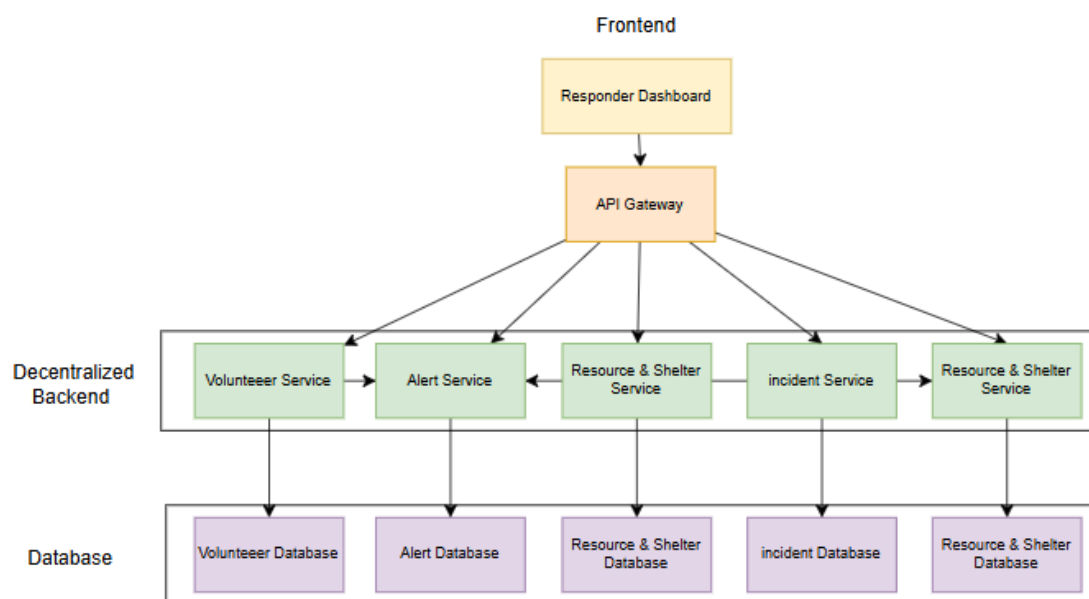
7,0 ARCHITECTURE DIAGRAM

7.1 SYSTEM CONTEXT DIAGRAM



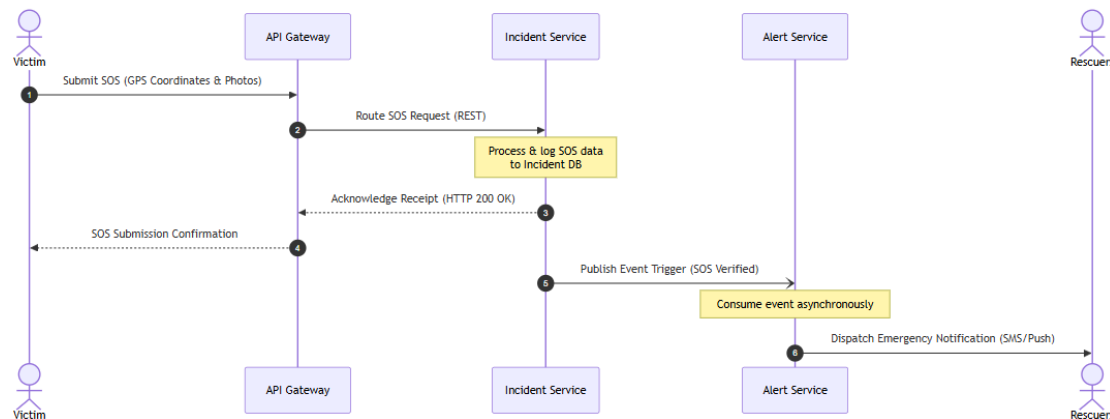
This diagram illustrates the high-level interactions between the Disaster Relief Coordination Platform (DRCP) and its external environment. It demonstrates how external users (Victims, Rescuers, Volunteers) and third-party systems (like Government Databases) securely exchange critical data with the platform during an emergency.

7.2 COMPENANT SERVICE ARCHITECTURE DIAGRAM



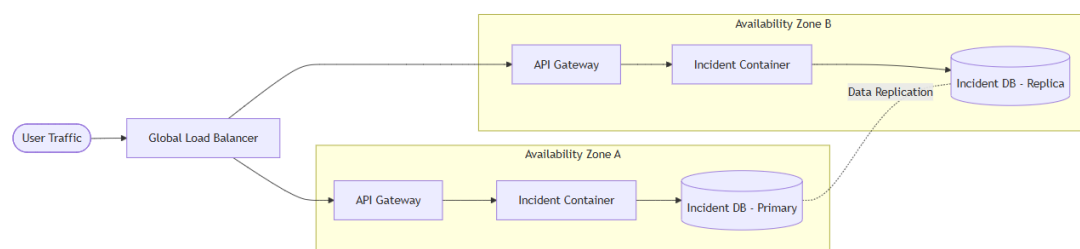
This diagram maps the internal structure of the DRCP, showing how the API Gateway routes traffic to isolated, single responsibility microservices. Crucially, it highlights the strict "database-per-service" data layer, ensuring that no single database failure can crash the entire system.

7.3 INTERACTION DIAGRAM



This sequence diagram outlines the exact internal communication flow for processing an emergency SOS submission. It highlights the use of synchronous REST API calls for initial routing and asynchronous event triggers to dispatch life-saving alerts without creating system bottlenecks.

7.4 DEPLOYMENT DIAGRAM



This diagram details the system's cloud infrastructure, showing how individual microservices are containerized using Docker. This containerized approach allows critical services to be deployed, updated, and scaled independently behind a load balancer to handle sudden traffic spikes.

8.0 TECHNOLOGY STACK

The Disaster Relief Coordination Platform (DRCP) uses Node.js as the backend technology to develop the microservices. Node.js is suitable for handling multiple requests efficiently and supports the development of scalable web applications.

MongoDB is used as the database because it stores data in a flexible document-based format. This allows the system to manage different types of disaster-related information, such as incident reports, volunteer profiles, shelter records, and resource inventories.

JSON (JavaScript Object Notation) is used as the standard format for data exchange between microservices. It provides a lightweight and easy-to-read structure for transmitting information through REST APIs.

9.0 PROTOTYPE IMPLEMENTATION

The prototype of the Disaster Relief Coordination Platform (DRCP) demonstrates the proposed microservice architecture through several independent services, including the API Gateway, User Management Service, Incident Service, Volunteer Service, Resource Service, Shelter Service, and Alert Service. Each service performs a specific function and maintains its own database to ensure reliability and service independence.

The services communicate using REST APIs and event-driven messaging to support efficient data exchange. Docker containers are used to deploy each service independently, allowing easier maintenance and scalability. The prototype shows how the system can support disaster response activities through effective coordination, resource management, and real-time communication.

10.0 ARCHITECTURE EVALUATION

The Microservice Architecture is suitable for the Disaster Relief Coordination Platform (DRCP) because it divides the system into several independent services, each responsible for a specific task. This approach improves reliability, as a failure in one service does not affect the operation of the entire platform. For example, if the Alert Service encounters a problem, other services such as Incident Reporting and Resource Management can continue functioning normally.

The architecture also supports scalability by allowing individual services to be scaled according to demand. During disaster situations, services that receive many requests can be expanded without affecting other parts of the system. In addition, the use of REST APIs and event-driven communication enables efficient data exchange between services and external systems.

Another advantage is maintainability. Since each service operates independently and manages its own database, updates and improvements can be made without disrupting the whole platform. Although microservices require additional effort

for deployment and monitoring, they provide a flexible, reliable, and scalable solution that is well suited for disaster relief coordination.

11.0 TEAM CONTRIBUTIONS

TEAM CONTRIBUTION	
Team Member	Core Responsibilities
MUHAMMAD HAZIM IHSAN	Engineered the core Incident Service and structured its dedicated database to securely manage emergency reports and isolate critical data.
PETER LOURDES FERNANDEZ	Developed the central API Gateway, handling request routing and seamlessly combining the independent microservices into a unified platform.
AHMAD SHAHIRAN AMSYAR BIN SHAFEIN	Built the Resource Service and its corresponding database to efficiently track, manage, and update the inventory of relief supplies.

12.0 CONCLUSION AND REFLECTION

CONCLUSION

In conclusion, the Disaster Relief Coordination Platform (DRCP) was designed using a Microservice Architecture to support efficient disaster response and management. The proposed architecture separates the system into independent services, improving reliability, scalability, and maintainability. Through the identification of services, communication design, and architectural diagrams, the project demonstrates how microservices can be applied to solve real-world disaster management challenges. Overall, the selected architecture provides a flexible and robust foundation for supporting emergency operations and future system expansion.

REFLECTION

This project provided valuable experience in analysing system requirements and designing a software architecture based on real-world scenarios. Through this assignment, we gained a better understanding of microservice architecture, service decomposition, communication mechanisms, and architectural design principles. We also learned the importance of scalability, fault tolerance, and maintainability when developing large-scale systems. Working as a team helped improve our communication, collaboration, and problem-solving skills throughout the project. Overall, this assignment enhanced our knowledge of software architecture and its practical applications in system development.